

Tarea 3.5 – Análisis Germinal y Somático con nf-core/sarek

Autora: Martha Flórez

Fecha: 07.12.2025

1. Introducción

El análisis de variantes germinales y somáticas es fundamental para comprender tanto la predisposición hereditaria como los mecanismos moleculares adquiridos por un tumor. En este trabajo se aplicó el pipeline **nf-core/sarek**, estándar en bioinformática clínica, para realizar:

- Llamado de variantes **germinales** mediante *HaplotypeCaller* (GATK).
- Llamado de variantes **somáticas** mediante *Mutect2* (GATK).
- Anotación funcional con **snpEff**.
- Filtrado de variantes no sinónimas con potencial impacto proteico.
- Clasificación en **OncoKB** (somáticas) y **gnomAD** (germinales).

Finalmente, se compararon ambos perfiles genómicos y se discutió su relevancia biológica y clínica.

2. Metodología

2.1. Estructura de trabajo

El proyecto se organizó en los siguientes directorios:

```
Sesion5/
├── code/
│   ├── sarek_germinal.sh
│   ├── sarek_somatic.sh
│   └── local_sarek_8cpus.config
├── data/
│   ├── R1.fastq.gz
│   └── R2.fastq.gz
└── results/
```

2.2 Activación del ambiente de sarek

```
pyenv activate sarek_taller-pyenv
```

2.3 Directorio de trabajo

Te recomendamos crear una carpeta de trabajo que tenga subdirectorios con los datos de input y la salida del pipeline sarek.

```
# Crear directorio de trabajo
mkdir pipeline_sarek
  code -> Almacena los script sarek_germinal.sh y sarek_somatic.sh
  results -> Directorio de salida del pipeline (acá se almacenará los
  resultados del pipeline)
mkdir results
```

Antes de editar el comando, copio los archivos `S7_R1.fastq.gz` y `S7_R2.fastq.gz` que se encuentran en `~/181004_curso_calidad_datos_NGS/fastq_raw`

```
cp S7_R1.fastq.gz S7_R2.fastq.gz
/home/bioinfo1/181004_curso_calidad_datos_NGS/fastq_raw/
```

Reemplazo los nombres de los archivos `S7_R1` y `S7_R2` como viene en el scrip

```
mv S7_R1.fastq.gz R1.fastq.gz
mv S7_R2.fastq.gz R2.fastq.gz
```

2.4 Scripts

Te presentamos un script para ejecutar el pipeline sarek para datos germinales.

A continuación te mostramos en qué consiste el script `sarek_germinal.sh`. El llamador de variantes que debes usar para variantes germinales es `haplotypcaller`.

Entra al directorio `code`

```
cd code
```

Luego, con un editor de texto ,por ej. nano, crear un archivo bash para ejecutar el modo germinal de sarek:

```

#!/bin/bash
# Ejecuta nf-core/sarek en modo GERMINAL para la muestra germinal S7.
# Uso:
#   A) Nombre de muestra automático (recomendado para alumnos):
#       bash sarek_germinal.sh R1.fastq.gz R2.fastq.gz /ruta/output
#   B) Nombre de muestra explícito:
#       bash sarek_germinal.sh R1.fastq.gz R2.fastq.gz /ruta/output
#       nombre_muestra
#
# El script crea internamente un samplesheet CSV como requiere nf-core/sarek
# y luego llama a:
#   nextflow run nf-core/sarek --input samplesheet.csv ...

if [ "$#" -lt 3 ] || [ "$#" -gt 4 ]; then
    echo "Uso: bash sarek_germinal.sh R1.fastq.gz R2.fastq.gz /ruta/output
[nombre_muestra]"
    exit 1
fi

R1=$1
R2=$2
OUT=$3

mkdir -p "$OUT"

# Si se entrega un cuarto argumento, se usa como nombre de muestra
if [ "$#" -eq 4 ]; then
    SAMPLE=$4
else
    # Detección automática del nombre de muestra desde R1
    base=$(basename "$R1")

    # Elimina sufijos comunes de R1
    sample=${base%%_R1.fastq.gz}
    sample=${sample%%_R1.fq.gz}
    sample=${sample%%_1.fastq.gz}
    sample=${sample%%_1.fq.gz}
    sample=${sample%%.fastq.gz}
    sample=${sample%%.fq.gz}

    SAMPLE=$sample
    echo "Detectado nombre de muestra automáticamente: ${SAMPLE}"
fi

# Intentar obtener rutas absolutas (si readlink -f está disponible)
if command -v readlink >/dev/null 2>&1; then
    R1_ABS=$(readlink -f "$R1")
    R2_ABS=$(readlink -f "$R2")
else
    R1_ABS="$R1"

```

```

R2_ABS="$R2"
fi

SHEET="${OUT}/samplesheet_germline_${SAMPLE}.csv"

echo "Creando samplesheet: $SHEET"
cat > "$SHEET" <<EOF
patient,sex,status,sample,lane,fastq_1,fastq_2
${SAMPLE},NA,0,${SAMPLE},L1,${R1_ABS},${R2_ABS}
EOF

echo "Lanzando nf-core/sarek en modo germinal..."
nextflow run nf-core/sarek \
  --input "$SHEET" \
  --genome GATK.GRCh38 \
  --outdir "$OUT" \
  --tools haplotypcaller \
  --profile singularity \
  -c /home/bioinfo1/korostica/test_tutorial/code/local_sarek_8cpus.config \
  --resume

```

Ejecutar el siguiente comando:

```

bash sarek_germinal.sh ../data/R1.fastq.gz ../data/R2.fastq.gz ../results S7
bash sarek_somatic.sh ../data/R1.fastq.gz ../data/R2.fastq.gz ../results S7

```

A continuación mostramos el script `sarek_somatic.sh` . El llamador de variantes mutect2.

```

#!/bin/bash
# Ejecuta nf-core/sarek en modo SOMÁTICO (tumor-only)S7.
# Uso:
#   A) Nombre de muestra automático:
#       bash sarek_somatic.sh R1.fastq.gz R2.fastq.gz /ruta/output
#   B) Nombre de muestra explícito:
#       bash sarek_somatic.sh R1.fastq.gz R2.fastq.gz /ruta/output
#       nombre_muestra
#
# El script crea internamente un samplesheet CSV como requiere nf-core/sarek.

if [ "$#" -lt 3 ] || [ "$#" -gt 4 ]; then
    echo "Uso: bash sarek_somatic.sh R1.fastq.gz R2.fastq.gz /ruta/output
[nombre_muestra]"
    exit 1
fi

R1=$1
R2=$2
OUT=$3

mkdir -p "$OUT"

# Si se entrega un cuarto argumento, se usa como nombre de muestra
if [ "$#" -eq 4 ]; then
    SAMPLE=$4
else
    base=$(basename "$R1")

    sample=${base%_R1.fastq.gz}
    sample=${sample%_R1.fq.gz}
    sample=${sample%_1.fastq.gz}
    sample=${sample%_1.fq.gz}
    sample=${sample%.fastq.gz}
    sample=${sample%.fq.gz}

    SAMPLE=$sample
    echo "Detectado nombre de muestra automáticamente: ${SAMPLE}"
fi

# Rutas absolutas
if command -v readlink >/dev/null 2>&1; then
    R1_ABS=$(readlink -f "$R1")
    R2_ABS=$(readlink -f "$R2")
else
    R1_ABS="$R1"
    R2_ABS="$R2"
fi

SHEET="${OUT}/samplesheet_somatic_${SAMPLE}.csv"

```

```

echo "Creando samplesheet: $SHEET"
cat > "$SHEET" <<EOF
patient,sex,status,sample,lane,fastq_1,fastq_2
${SAMPLE},NA,1,${SAMPLE},L1,${R1_ABS},${R2_ABS}
EOF

echo "Lanzando nf-core/sarek en modo somático (tumor-only)..."
nextflow run nf-core/sarek \
  --input "$SHEET" \
  --genome GATK.GRCh38 \
  --outdir "$OUT" \
  --tools mutect2 \
  --profile singularity \
  -c /home/bioinfo1/korostica/test_tutorial/code/local_sarek_8cpus.config \
  -resume

```

Ejecutar el siguiente comando:

```
bash sarek_somatic.sh R1.fastq.gz R2.fastq.gz ../results
```

Los dos script son capaces de:

- Detecta nombre_muestra desde R1 si no se lo das
- Crea samplesheet_germline_.csv en el outdir
- Llama a Sarek con ese CSV

Debemos crear un archivo de configuración para indicarle a nextflow la capacidad de memoria que debe utilizar. Para esto creamos el archivo `local_sarek_8cpus.config` que debe estar en el directorio `code`.

```
// Config local para correr Sarek en un servidor con 16 CPUs

executor {
    name = 'local'
    cpus = 16      // lo que realmente tiene tu maquina
}

// Limite global: ningun proceso puede usar mas de 8 CPUs
process {
    resourceLimits = [ cpu: 8 ]
}

// (opcional, pero mas explicito)
// Forzar especificamente el proceso de alineamiento BWAMEM1_MEM a 8 CPUs
process {
    withName:
    'NFCORE_SAREK:SAREK:FASTQ_PREPROCESS_GATK:FASTQ_ALIGN:BWAMEM1_MEM' {
        cpus = 8
    }
}
```

2.5 Asignar permisos de ejecución

Una vez que se haya creado ambos script debes darles permisos de ejecución.

```
# Estando dentro del directorio code
chmod +x sarek_germinal.sh
chmod +x sarek_somatic.sh
```

2.6 Ejecución

Para correr ambos scripts debes ejecutar el siguiente código entregando como parámetro la ruta del read 1 y read 2 y el directorio de salida. Reemplaza los nombres de archivos de acuerdo a lo necesario.

El script germinal se ejecuta de la siguiente forma:

```
bash sarek_germinal.sh ../data/R1.fastq.gz ../data/R2.fastq.gz ../results
```

El script Somático se ejecuta de la siguiente forma:

```
bash sarek_somatic.sh ../data/R1.fastq.gz ../data/R2.fastq.gz ../results
```

Nota: muestre los comandos usados y desde qué directorio se ejecutaron en su informe.

Trabajo práctico: Análisis germinal y somático con nf-core/sarek + interpretación en OncoKB y gnomAD

```
Sesion5/
├── code/
│   ├── sarek_germinal.sh
│   ├── sarek_somatic.sh
│   └── local_sarek_8cpus.config
├── data/
│   ├── R1.fastq.gz
│   └── R2.fastq.gz
└── results/
    ├── samplesheet_germline_R1.csv
    └── samplesheet_somatic_R1.csv
```

1. Objetivos de la actividad

En este trabajo práctico aplicarás el pipeline **nf-core/sarek** para:

1. Ejecutar un **análisis germinal** (variantes constitucionales) a partir de un FASTQ.
2. Ejecutar un **análisis somático** (variantes adquiridas en el tumor) usando la misma muestra tumoral.
3. Comparar cuantitativamente los resultados germinales vs somáticos.
4. Realizar una **búsqueda e interpretación de variantes** usando:
 - **OncoKB** (base de conocimiento oncológica somática).
 - **gnomAD** (frecuencias poblacionales germinales).
5. Elaborar un **informe corto** discutiendo las diferencias observadas y la posible relevancia biológica/clínica de las variantes seleccionadas obtenidas a apartir del análisis germinal versus el somático.

2. Datos y recursos

- Conjunto de datos (FASTQ) **Cada estudiante analizará su muestra.**
- Pipeline: **nf-core/sarek** (versión indicada en el tutorial).
- Recursos online:
 - <https://www.oncokb.org>

- <https://gnomad.broadinstitute.org>
-

3. Análisis germinal y somático con sarek

3.1. Ejecución del pipeline germinal y somático

1. Ejecutar **sarek** primero en modo germinal y luego en somático de la muestra provista:

```
# GERMINAL
bash sarek_germinal.sh ../data/R1.fastq.gz ../data/R2.fastq.gz ../results
# SOMATICO
bash sarek_somatic.sh ../data/R1.fastq.gz ../data/R2.fastq.gz ../results
```

3.2. Resultados esperados

Se trabajó con dos conjuntos de variantes:

Variantes germinales

- Llamadas a partir de **HaplotypeCaller (GATK)**.
- Representan variantes heredadas presentes en todas las células.

Variantes somáticas

- Llamadas a partir de **Mutect2 (GATK)**.
- Representan variantes adquiridas específicas del tumor.

Luego, se procede a:

- revisar VCF de variantes germinales y somáticas detectadas
- Reportes de calidad (MultiQC).

```

Sesion5/results/
├── variant_calling/
│   ├── haplotypcaller/R1/ # GERMINAL
│   │   ├── R1.haplotypcaller.filtered.vcf.gz
│   │   ├── R1.haplotypcaller.filtered.vcf.gz.tbi
│   │   ├── R1.haplotypcaller.vcf.gz
│   │   └── R1.haplotypcaller.vcf.gz.tbi
│   └── mutect2/R1/ # SOMATICO
│       ├── R1.mutect2.artifactprior.tar.gz
│       ├── R1.mutect2.contamination.table
│       ├── R1.mutect2.filtered.vcf.gz
│       ├── R1.mutect2.filtered.vcf.gz.filteringStats.tsv
│       ├── R1.mutect2.filtered.vcf.gz.tbi
│       ├── R1.mutect2.pileups.table
│       ├── R1.mutect2.segmentation.table
│       ├── R1.mutect2.vcf.gz
│       ├── R1.mutect2.vcf.gz.stats
│       └── R1.mutect2.vcf.gz.tbi
└── multiqc$
    ├── multiqc_data
    ├── multiqc_plots
    └── multiqc_report.html

```

Contar variantes totales germinales

```

module load bcftools
bcftools --version
# Número total de variantes germinales filtradas
bcftools view -H \
    results/variant_calling/haplotypcaller/R1/R1.haplotypcaller.filtered.vcf.gz \
    | wc -l
145
# Número total de variantes somáticas filtradas
bcftools view -H \
    results/variant_calling/mutect2/R1/R1.mutect2.filtered.vcf.gz \
    | wc -l
227

```

4. Obtención de variantes

Archivo base utilizado:

- R1.haplotypcaller.filtered.vcf.gz

- `R1.mutect2.filtered.vcf.gz`

Ambos archivos fueron previamente **filtrados por GATK** para conservar solo variantes de alta confianza (`PASS`).

5. Anotación funcional de variantes con snpEff

Tanto las variantes germinales como las somáticas fueron anotadas usando:

- **Herramienta:** `snpEff`
- **Genoma de referencia:** `GRCh38.86`
- **Objetivo:** Asignar a cada variante su:
 - Gen afectado
 - Tipo de efecto molecular
 - Impacto funcional (missense, frameshift, stop gained, etc.)

Debido a limitaciones de memoria del servidor, se asignó memoria adicional a Java: `export _JAVA_OPTIONS="-Xmx4g"` . Luego se ejecutó snpEff en ambos casos:

Germinal:

```
snpEff -v GRCh38.86 R1.haplotypcaller.filtered.vcf.gz >
R1.haplotypcaller.filtered.ann.vcf
```

Somático:

```
snpEff -v GRCh38.86 R1.mutect2.filtered.vcf.gz >
R1.mutect2.filtered.ann.vcf
```

6. Filtrado de variantes no sinónimas (impacto funcional)

Posteriormente, tanto en germinal como en somático, se filtraron únicamente las variantes con **posible impacto funcional sobre la proteína**, utilizando `bcftools` .

Criterio de filtrado aplicado

Se seleccionaron variantes cuyo campo `ANN` de `snpEff` contuviera alguno de los siguientes efectos:

- `missense_variant`
- `stop_gained`
- `frameshift`
- `splice`

Germinal no sinónimas

```
bcftools view \  
-i 'INFO/ANN~"missense_variant|stop_gained|frameshift|splice" \  
R1.haplotypcaller.filtered.ann.vcf.gz \  
-Ov -o germinal_nonsyn.vcf
```

Somáticas no sinónimas

```
bcftools view \  
-i 'INFO/ANN~"missense_variant|stop_gained|frameshift|splice" \  
R1.mutect2.filtered.ann.vcf.gz \  
-Ov -o somatic_nonsyn.vcf
```

Estos archivos contienen **solo las variantes con mayor probabilidad de afectar la función proteica**, y son los utilizados para el análisis biológico posterior.

El resultado final son dos conjuntos depurados:

```
germinal_nonsyn.vcf  
somatic_nonsyn.vcf`
```

Estos archivos representan la base para el **análisis clínico y biológico posterior** del estudio.

7. Comparación germinal vs somático

Para la comparación utilizamos los siguientes datos generados:

snpEff_germinal_summary.html
snpEff_somatica_summary.html

TIPO DE VARIANTE	Germinal	Somática
Número total de variantes	145	227
Variantes conocidas (con ID)	63 (43.4 %)	0 (0 %)
SNPs	107	188
Inserciones (INS)	11	22
Deleciones (DEL)	27	12
MNPs	0	5
Impacto ALTO (HIGH)	41 (4.57 %)	54 (3.56 %)
Impacto MODERADO (MODERATE)	75 (8.36 %)	148 (9.74 %)
Impacto BAJO (LOW)	90 (10.03 %)	196 (12.90 %)
MODIFIER (no codificante principalmente)	691 (77.0 %)	1121 (73.8 %)
Missense	75 (50.3 %)	144 (57.6 %)
Nonsense (STOP gain)	8 (5.37 %)	14 (5.6 %)
Silenciosas (Synonymous)	66 (44.3 %)	92 (36.8 %)
Frameshift	32	28
Splice (todas)	31	125
Splice acceptor	1	10
Splice donor	0	2
Splice region	30	113
STOP gained	12	19
Cromosoma con más variantes	Chr 13 (28)	Chr 13 (49)
Ts/Tv (SNPs)	3.88	2.42

8. Búsqueda en OncoKB y gnomAD

Los siguientes comandos fueron utilizados para filtrar las variantes de mayor impacto tanto variantes germinales como somáticas.

8.1. Germinales

```
$ zcat R1.haplotypcaller.filtered.ann.vcf.gz \  
> | grep 'rs' \  
> | grep 'missense_variant' \  
> > R1_filtered_missense_rs.ann.vcf
```

Genera el archivo `R1_filtered_missense_rs.ann.vcf`

#	Cromosoma	Posición	rsID	REF	ALT	Gen	Tipo
1	chr1	43,337,870	rs572208458	A	G	MPL	Missense
2	chr8	78,738,511	rs762037062	G	A	IL7	Missense
3	chr9	36,840,626	rs3780135	G	A	PAX5	Missense
4	chr9	130,873,004	rs121913457	T	C	ABL1	Missense
5	chr13	28,050,157	rs1933437	G	A	FLT3	Missense
6	chr13	32,355,250	rs169547	T	C	BRCA2	Missense
7	chr19	12,943,967	rs1049481	G	T	CALR	Missense

Germinales → gnomAD

Gen	Variante p.	rsID	Frecuencia gnomAD	Interpretación
MPL	Met8Val	rs572208458	0.005%	Ultra-rara, VUS
IL7	Thr118Ile	rs762037062	0.0006%	Extremadamente rara, VUS
PAX5	Thr264Ile	rs3780135	80–90%	Variante benigna
ABL1	Met370Thr	rs121913457	Muy baja	Reportada patogénica en síndrome germinal
FLT3	Thr227Met	rs1933437	54%	Benigna
BRCA2	Val2466Ala	rs169547	98%	Benigna
CALR	Gly159Cys	rs1049481	Alta	Polimorfismo común

La mayoría de las variantes germinales identificadas (en PAX5, FLT3, BRCA2 y CALR) presentan frecuencias alélicas muy altas en población general, y están clasificadas en ClinVar como variantes benignas o polimorfismos comunes, por lo que no se consideran responsables de un fenotipo monogénico de alto riesgo.

En contraste, variantes como **MPL p.Met8Val** y **IL7 p.Thr118Ile** son extremadamente raras en gnomAD y suelen clasificarse como variantes de significado incierto (VUS), mientras que **ABL1 p.Met370Thr** es una variante germinal muy rara con descripciones previas como patogénica/likely pathogenic en un síndrome de malformaciones. Estas variantes requieren interpretación cuidadosa en relación con el fenotipo clínico y otras evidencias (funcionales, segregación, etc.).

8.2 Somáticas

El archivo `R1_somatic_missense_all.ann.vcf` podemos armar una tabla resumen muy clara usando el **primer efecto ANN de cada variante** (el missense principal).

Aquí está la tabla (10 variantes):

#	Cromosoma	Posición	REF>ALT	Gen	Consecuencia (ANN)	Imp
1	chr1	43 337 870	A>G	MPL	missense_variant	MOD
2	chr2	197 400 774	T>C	SF3B1	missense_variant	MOD
3	chr2	197 408 098	G>A	SF3B1	missense_variant	MOD
4	chr4	54 726 014	G>A	KIT	missense_variant	MOD
5	chr4	54 728 011	C>T	KIT	missense_variant & splice_region_variant	MOD
6	chr5	150 117 648	C>T	PDGFRB	missense_variant	MOD
7	chr5	150 119 480	C>T	PDGFRB	missense_variant	MOD
8	chr5	150 129 952	G>A	PDGFRB	missense_variant	MOD
9	chr7	50 400 308	T>C	IKZF1	missense_variant	MOD
10	chr7	140 734 740	A>G	BRAF	missense_variant	MOD

Primero, contexto general (igual que para la tabla anterior):

OncoKB **sí incluye estos genes** (MPL, SF3B1, KIT, PDGFRB, IKZF1, BRAF) como genes accionables u oncológicos, pero sólo **algunos hotspots específicos** (p.ej. MPL W515, SF3B1 K700E/R625, KIT D816, PDGFRB fusions, IKZF1 deleciones, BRAF V600), no cualquier missense rara.

La lista mostrada corresponde a variantes **somáticas no-hotspot** → en OncoKB, como regla general, se manejan como “*other missense / unknown significance*” si es que aparecen, y muchas ni siquiera están listadas individualmente.

```
zcat R1.mutect2.filtered.ann.vcf \
| grep 'rs' \
| grep 'missense_variant' \
> R1_filtered_missense_rs.ann.vcf
```

Genera el archivo `R1_somatic_missense.ann.vcf`

```
cat R1.mutect2.filtered.ann.vcf | grep 'missense_variant' | grep -E
'BRCA1|BRCA2|KRAS|TP53|VEGFA|BRAF|JAK1|JAK2|JAK3' > R1_somatic_missense.ann.vcf
```

#	Cromosoma	Posición	REF>ALT	Gen	Consecuencia	Impac
1	chr7	140734740	A>G	BRAF	missense_variant	MODER
2	chr7	140781580	C>A	BRAF	missense_variant	MODER
3	chr7	140800369	A>G	BRAF	missense_variant	MODER
4	chr7	140800371	T>A	BRAF	missense_variant	MODER
5	chr9	5 078 377	G>C	JAK2	missense_variant	MODER
6	chr9	5 089 777	T>A	JAK2	missense_variant	MODER
7	chr9	5 090 838	G>A	JAK2	missense_variant	MODER
8	chr9	5 090 845	C>A	JAK2	missense_variant	MODER
9	chr12	25 245 326	G>A	KRAS	missense_variant	MODER
10	chr13	32 337 226	T>A	BRCA2	missense_variant	MODER

6.1. Somáticas → OncoKB

Gen	Variantes encontradas	Situación en OncoKB
BRAF	S720P, W476C, S325P, D324V	No-V600, no hotspot → VUS
JAK2	K688N, L892Q, G996R, T998N	No-V617F, no oncogenicidad definida
KIT	A502T, P627L	No hotspots exón 11/17
PDGFRB	P462S, A929T, G1036D	No fusión ni hotspot; VUS
IKZF1	L414P	No hotspot, ICUS/VUS
KRAS	T20M	Reportada pero sin evidencia clínica, no G12/G13e).

Todas las variantes identificadas son cambios missense de impacto moderado en genes clásicos de cáncer (BRAF, JAK2, KRAS y BRCA2). Sin embargo, ninguna corresponde a los hotspots canónicos ampliamente descritos (por ejemplo, BRAF V600E, KRAS G12C o JAK2 V617F), y al contrastarlas con OncoKB no se encuentran clasificadas como mutaciones claramente oncogénicas o clínicamente accionables. La mayoría aparecen en bases como ClinVar o el Allele Registry como variantes reportadas, muchas de ellas con significado clínico incierto (VUS). Por lo tanto, su relevancia biológica y clínica debe interpretarse con cautela y, en ausencia de evidencia funcional o clínica adicional, se consideran variantes sin clasificación definitiva.

9. Discusión

El análisis reveló diferencias claras entre los perfiles germinal y somático:

1. **El exoma somático presenta mayor complejidad**, con más variantes totales y más variantes de impacto moderado/alto.
2. **Las variantes germinales** corresponden principalmente a **polimorfismos benignos**, según gnomAD y ClinVar.
3. En contraste, **las variantes somáticas** afectan genes oncológicos relevantes (BRAF, JAK2, KIT, KRAS, PDGFRB, IKZF1), pero **ninguna corresponde a mutaciones definidas como oncogénicas ni accionables según OncoKB**.
4. Esto sugiere que el tumor analizado no presenta mutaciones driver clásicas, o bien que las mutaciones presentes aún no cuentan con evidencia funcional suficiente.

5. La alta proporción de variantes de splice y missense en somático podría sugerir inestabilidad genómica moderada, típica de tumores sin un driver dominante.
-

10. Conclusiones

- El pipeline **nf-core/sarek** permitió realizar un flujo de análisis robusto de variantes germinales y somáticas.
- Las variantes germinales evaluadas en **gnomAD** corresponden principalmente a polimorfismos benignos o VUS ultra-raros.
- Las variantes somáticas no corresponden a mutaciones accionables según **OncoKB**, por lo que actualmente se consideran **VUS**.
- El análisis integrador evidencia la importancia de combinar herramientas bioinformáticas, anotadores funcionales y bases clínicas para interpretar variantes.